PRAKTIKUM 2
MEMORI DINAMIS

NAMA: Ihsanta Zaki Sanjaya

KELAS: 1 D4 Teknik Komputer B

NRP: 3223600058

DOSEN: Dr Bima Sena Bayu Dewantara S.ST, M.T

MATA KULIAH: Praktikum Pemograman Dasar 2

TANGGAL: Rabu, 13 Maret 2024

D4 TEKNIK KOMPUTER
POLITEKNIK ELEKTRONIKA NEGERI SURABAYA
2024

Praktikum 2
Memori Dinamis

Untuk memberikan pemahaman kepada anda tentang konsep dasar memori dinamis dan cara menggunakan dalam program C++, maka cobalah beberapa modul percobaan dibawah ini.

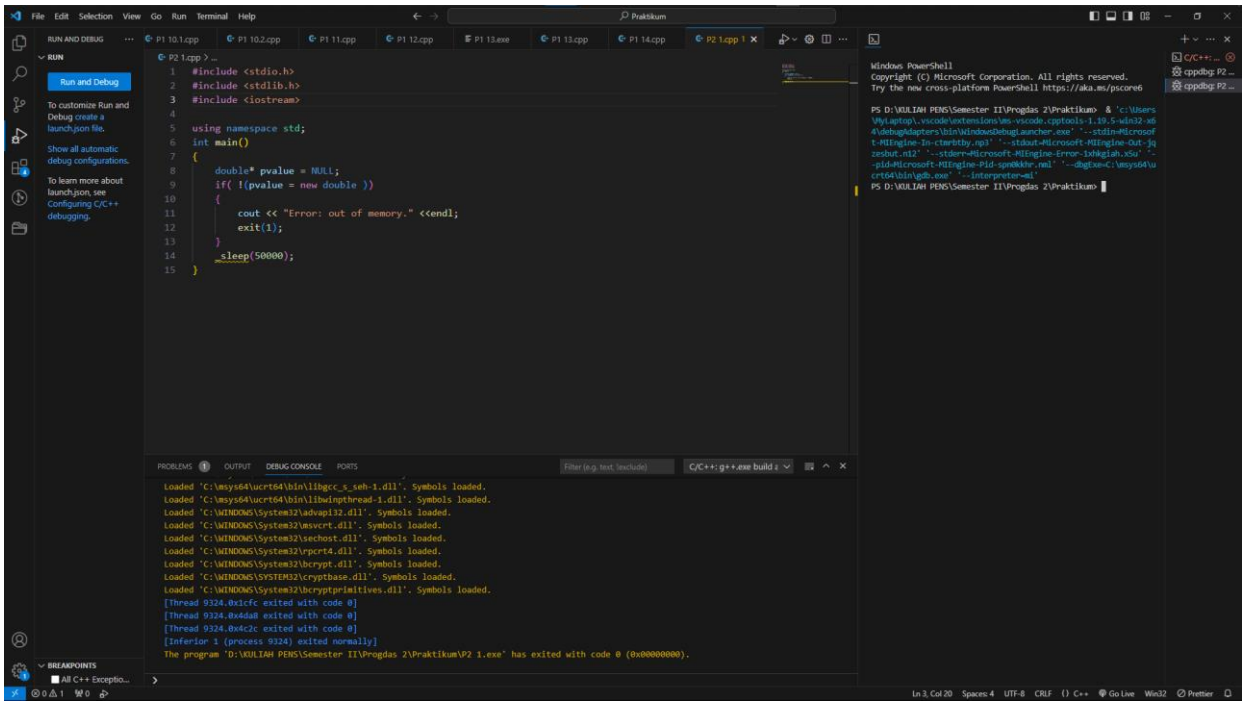
1. **Percobaan 1 – Inisialisasi variabel menggunakan operator *new* untuk alokasi memori dinamis**

   Kita dapat mendefinisikan sebuah pointer ke tipe double dan kemudian meminta memori untuk dialokasikan pada waktu eksekusi. Kita dapat melakukannya dengan menggunakan operator ***new*** seperti contoh di bawah ini.

   Terkadang alokasi memori tidak dapat selalu sukses dilakukan, jika free store telah habis. Untuk mengeceknya, sebuah operator ***new*** dengan mengembalikan pointer bernilai NULL dapat dilakukan sebagai berikut:

```
#include    <stdio.h>
#include   <stdlib.h>
#include <iostream>

using namespace std;

void main()
{
        double* pvalue = NULL;
        if( !(pvalue = new double ))
        {
                cout << "Error: out of memory." <<endl;
                exit(1);
        }

        _sleep(50000);
}
```

Hasil percobaan:



Dari percobaan diatas, apakah yang dapat anda simpulkan? Jelaskan!

Program melakukan alokasi memori dinamis dengan operator new. Setelah deklarasi pointer dan alokasi memori, program tidur sementara selama 50 detik menggunakan fungsi _sleep(). Namun, program tidak melakukan pembebasan memori yang tepat, yang dapat menyebabkan kebocoran memori. Dengan menggunakan operator delete untuk membebaskan memori yang dialokasikan dinamis sebelum keluar dari program.

Dari percobaan pertama, dapat disimpulkan bahwa:

- Terdapat kesalahan alokasi memori, dengan gagalnya pengalokasian memori dinais pada variabel ‘pvalue’, sehingga dari pesan “Error:out of memory”. Program tidak dapat melanjutkan eksekusi karena tidak adanya memori yang tersedia untuk alokasi.

- Dikarenakan gagalnya alokasi memori tersebut, program mencetak pesan kesalahan dan keluar menggunakan fungsi ‘exit()’ dengan status keluar1. Oleh karena itu tidak ada pembebasan memori yang dilakukan, sehingga dapat meyebabkan kebocoran memori (jika program lebuh kompleks)

Petingnya penanganan kesalahan alokasi memori dalam program, serta pentingnya pembebasan memori yang tepat untuk mencegah adanya kebocoran memori.

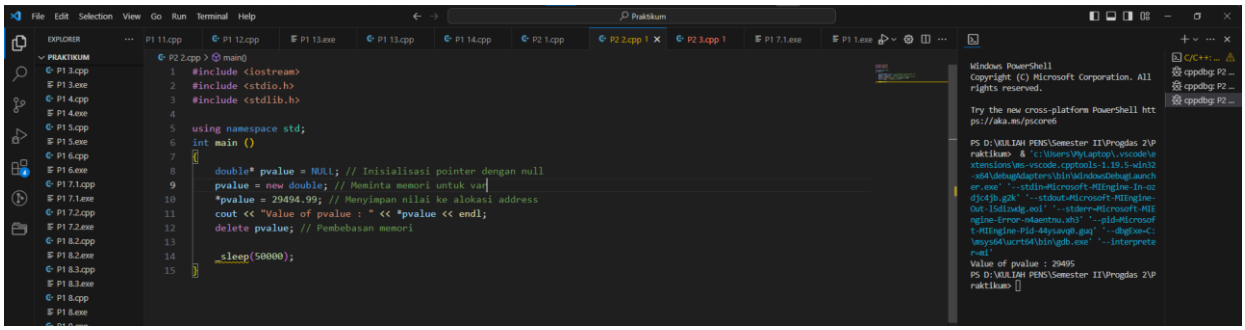
2. **Percobaan 2 – Membebaskan memori yang telah dialokasikan menggunakan operator *delete***

   Pada suatu saat, jika sebuah variabel yang dialokasikan secara dinamis tidak diperlukan lagi, maka anda dapat melakukan pembebasan memori tersebut yang menempati free

store dengan operator ***delete*** seperti contoh di bawah ini. Kita akan melihat bagaimana pasangan operator ***new*** dan ***delete*** bekerja.

```
#include <iostream>
#include <stdio.h>
#include <stdlib.h>

using namespace std;

void main ()
{
        double* pvalue = NULL; // Pointer initialized with null
        pvalue = new double;  // Request memory for the variable
        *pvalue = 29494.99;  // Store value at allocated address
        cout << "Value of pvalue : " << *pvalue << endl;
        delete pvalue;  // free up the memory.

        _sleep(50000);
}
```

Hasil percobaan:



Dari percobaan diatas, apakah yang dapat anda simpulkan? Jelaskan!

Program melakukan alokasi memori dinamis untuk sebuah variabel bertipe double menggunakan operator new. Nilai tersebut disimpan ke dalam lokasi memori yang dialokasikan sebelum dicetak ke layar. Setelah itu, memori yang dialokasikan dibebaskan menggunakan operator delete. Seperti percobaan pertama, program juga menggunakan fungsi _sleep() untuk menunda eksekusi selama 50 detik.

Dari percobaan kedua, dapat disimpulkan bahwa:

- Program berjalan dengan baik. Dengan kode keluar 0, menandakan bahwa tidak ada kesalahan yang fatal terjadi selama eksekusi progam.

- Program berhasil melakukan alokasi memori dinamis pada variabel ‘pvalue’ dan menetapkan nilai 29494.99 ke lokasi yang dialokasikan tadi. Sehingga nilai tersebut berhasil dicetak dengan benar sebelum program selesai.

- Setelah digunakan, memori yang dialokasikan secara dinamis dibebaskan dengan operator ‘delete’ untuk mencegah kebocoran memori.

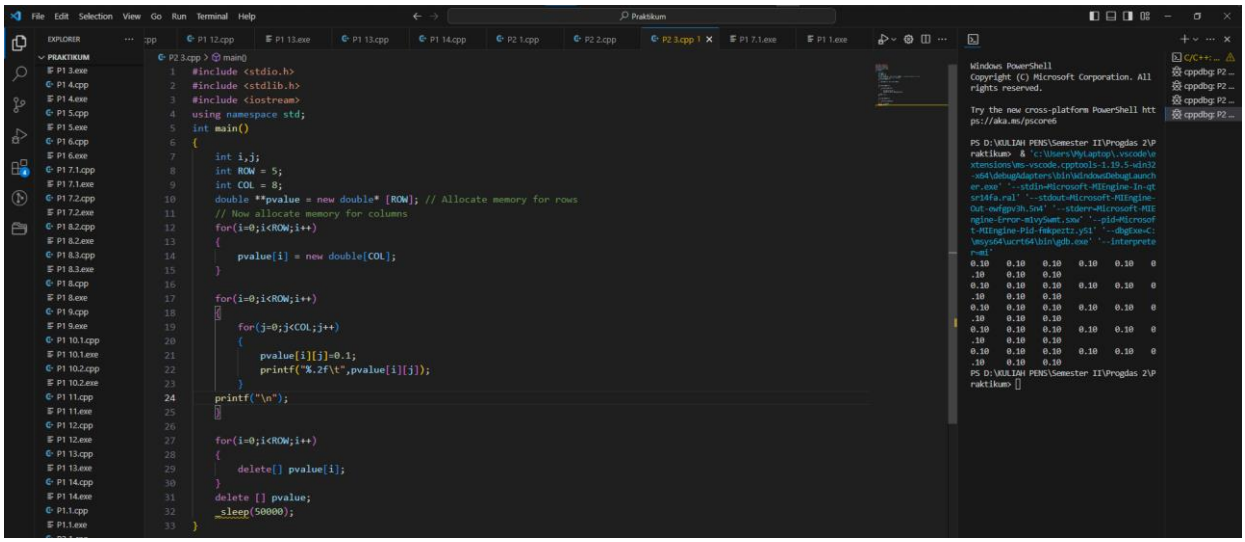
Program berhasil melakukan alokasi memori dinamis, menyimpan nilai ke dalamnya, mencetak nilai tersebut, dan kemudian membebaskan memori yang dialokasikan dengan benar sebelum program selesai dijalankan.

3. Percobaan 3 – Alokasi memori dinamis untuk array

Kita juga dapat mengalokasikan memori untuk sebuah array, contohnya bertipe double seperti di bawah ini. Dengan menggunakan sintaks yang sama, kita dapat mengalokasikan memori seperti contoh di bawah ini:

```
#include    <stdio.h>
#include   <stdlib.h>
#include <iostream>

using namespace std;

void main()
{
	int i,j;
	int ROW = 5;
	int COL = 8;
	double **pvalue = new double* [ROW]; // Allocate memory for rows
	// Now allocate memory for columns
	for(i=0;i<ROW;i++)
	{
		pvalue[i] = new double[COL];
	}

	for(i=0;i<ROW;i++)
	{
		for(j=0;j<COL;j++)
		{
			pvalue[i][j]=0.1;
			printf("%.2f\t",pvalue[i][j]);
		}
		printf("\n");
	}

	for(i=0;i<ROW;i++)
	{
		delete[] pvalue[i];
	}
	delete [] pvalue;

	_sleep(50000);
}
```

Hasil percobaan:



```
#include <stdio.h>
#include <stdlib.h>
#include <iostream>
using namespace std;
int main()
{
    int i,j;
    int ROW = 5;
    int COL = 8;
    double **pvalue = new double* [ROW]; // Allocate memory for rows
    // Now allocate memory for columns
    for(i=0;i<ROW;i++)
    {
        pvalue[i] = new double[COL];
    }

    for(i=0;i<ROW;i++)
    {
        for(j=0;j<COL;j++)
        {
            pvalue[i][j]=0.1;
            printf("%.2f\t",pvalue[i][j]);
        }
    printf("\n");
    }

    for(i=0;i<ROW;i++)
    {
        delete[] pvalue[i];
    }
    delete [] pvalue;
    _sleep(50000);
}
```

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Try the new cross-platform PowerShell https://aka.ms/pscore6

PS D:\KULIAH PENS\Semester II\Progdas 2\Praktikum>
0.10    0.10    0.10    0.10    0.10    0.10
0.10    0.10    0.10    0.10    0.10    0.10
0.10    0.10    0.10    0.10    0.10    0.10
0.10    0.10    0.10    0.10    0.10    0.10
0.10    0.10    0.10    0.10    0.10    0.10
PS D:\KULIAH PENS\Semester II\Progdas 2\Praktikum>
```

Dari percobaan diatas, apakah yang dapat anda simpulkan? Jelaskan!

Program membuat matriks dua dimensi (array 2D) dengan alokasi memori dinamis. Pertama, variabel `i` dan `j` digunakan sebagai penghitung dalam loop, sedangkan `ROW` dan `COL` menentukan ukuran matriks. Selanjutnya, memori dialokasikan untuk array pointer `pvalue` dengan ukuran `ROW`, dan untuk setiap baris, memori dialokasikan untuk array kolom dengan ukuran `COL`. Setelah alokasi memori, matriks diinisialisasi dengan nilai `0.1` dan dicetak ke layar menggunakan `printf()` dengan format dua angka di belakang koma. Setelah digunakan, memori yang dialokasikan untuk setiap baris dan untuk array pointer `pvalue` dibebaskan menggunakan operator `delete[]` untuk mencegah kebocoran memori. Program juga menggunakan fungsi `_sleep()` seperti percobaan sebelumnya untuk menunda eksekusi selama 50 detik.

Dari percobaan ketiga, dapat disimpulkan bahwa:

- Program berjalan dengan baik. Dengan kode keluar 0, menandakan bahwa tidak ada kesalahan yang fatal terjadi selama eksekusi progam

- Matriks yang dicetak menunjukan bahwa matriks 5x8 berhasil diinisialisasi dengan nilai ‘0.10’. Setiap elemen matriks dicetak dalam satu baris dengan nilai yang dipisahkan oleh tab (\t), dan setiap baris baru ditandai dengan karakter baris baru (\n).

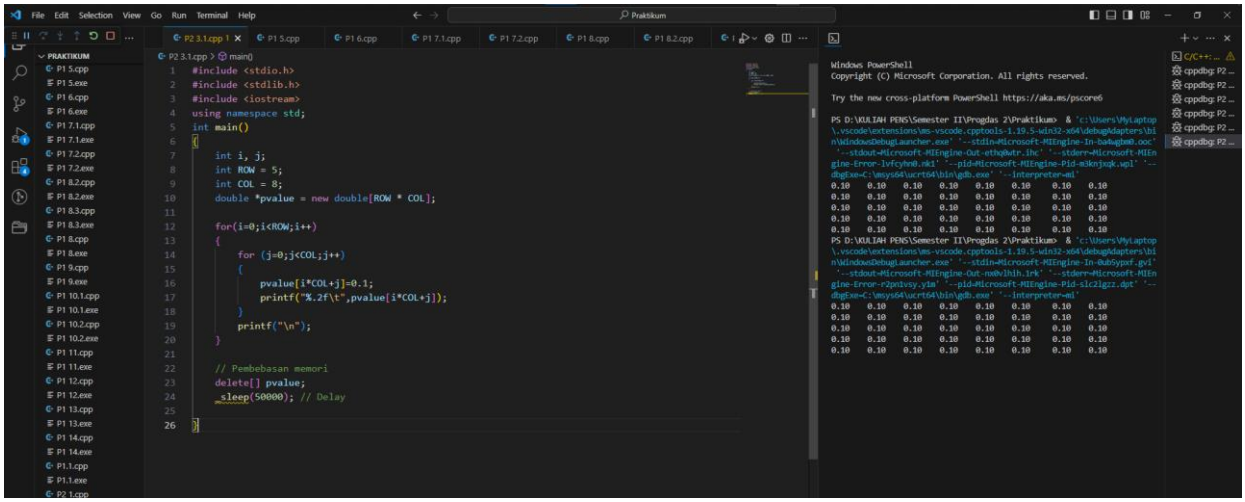
- Setelah digunakan, memori yang dialokasikan secara dinamis dibebaskan dengan operator ‘delete’ untuk mencegah kebocoran memori.

Ubahlah program diatas untuk array 1 dimensi!

Kode pemograman:

```
#include <stdio.h>
#include <stdlib.h>
#include <iostream>
using namespace std;
int main()
{
    int i, j;
    int ROW = 5;
    int COL = 8;
    double *pvalue = new double[ROW * COL];

    for(i=0;i<ROW;i++)
    {
        for (j=0;j<COL;j++)
        {
            pvalue[i*COL+j]=0.1;
            printf("%.2f\t",pvalue[i*COL+j]);
        }
        printf("\n");
    }

    // Pembebasan memori
    delete[] pvalue;
    _sleep(50000); // Delay

}
```

Hasil:

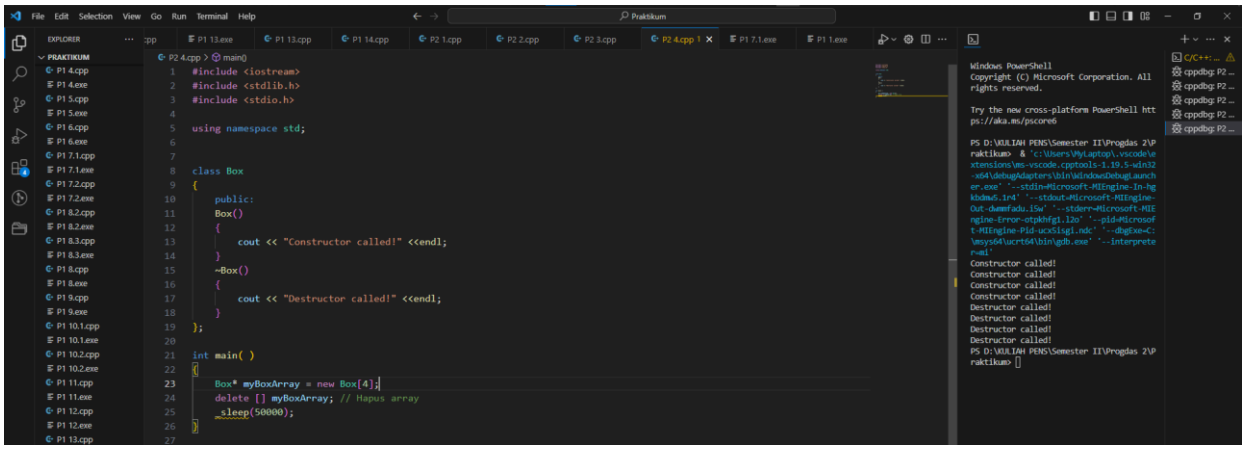


4. **Percobaan 4 – Alokasi memori dinamis untuk obyek**

   Obyek tidak berbeda dari tipe data yang sederhana. Contohnya dengan menggunakan array dari obyek seperti di bawah ini:

```
#include <iostream>
#include <stdlib.h>
#include <stdio.h>

using namespace std;

class Box
{
	public:
	Box()
	{
		cout << "Constructor called!" <<endl;
	}
	~Box()
	{
		cout << "Destructor called!" <<endl;
	}
};

void main( )
{
	Box* myBoxArray = new Box[4];

	delete [] myBoxArray; // Delete array

	_sleep(50000);
}
```

Hasil percobaan:



```
#include <iostream>
#include <stdlib.h>
#include <stdio.h>

using namespace std;


class Box
{
    public:
    Box()
    {
        cout << "Constructor called!" <<endl;
    }
    ~Box()
    {
        cout << "Destructor called!" <<endl;
    }
};

int main( )
{
    Box* myBoxArray = new Box[4];
    delete [] myBoxArray; // Hapus array
    _sleep(50000);
}
```

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Try the new cross-platform PowerShell https://aka.ms/pscore6

PS D:\KULIAH PENS\Semester II\Progdas 2\Praktikum>  & 'c:\Users\MyLaptop\.vscode\extensions\ms-vscode.cpptools-1.19.5-win32-x64\debugAdapters\bin\WindowsDebugLauncher.exe' '--stdin=Microsoft-MIEngine-In-hgkbdms5.1rd' '--stdout=Microsoft-MIEngine-Out-dwmmfadu.1Sw' '--stderr=Microsoft-MIEngine-Error-otpkhfg1.l2o' '--pid=Microsoft-MIEngine-Pid-ucx5isgi.ndc' '--dbgExe=C:\msys64\ucrt64\bin\gdb.exe' '--interpreter=mi'
Constructor called!
Constructor called!
Constructor called!
Constructor called!
Destructor called!
Destructor called!
Destructor called!
Destructor called!
PS D:\KULIAH PENS\Semester II\Progdas 2\Praktikum>
```

Dari percobaan diatas, apakah yang dapat anda simpulkan? Jelaskan!

Program menggunakan konstruktor dan destruktor dalam kelas C++. Kelas Box memiliki konstruktor yang mencetak pesan "Constructor called!" saat objek dibuat, dan destruktor yang mencetak pesan "Destructor called!" saat objek dihancurkan. Fungsi main() menggunakan operator new[] untuk membuat array dari empat objek Box. Setelah array digunakan, operator delete[] digunakan untuk membebaskan memori yang dialokasikan untuk array tersebut, yang menyebabkan pemanggilan destruktor untuk setiap objek dalam array.

Dari percobaan keempat, dapat disimpulkan bahwa:

- Konstruktor dipanggil ketika objek dibuat, terdapat pesan "Constructor called!" untuk setiap objek dalam array saat objek tersebut dibuat. Ini menunjukkan bahwa konstruktor kelas Box dipanggil dengan benar saat objek dibuat.

- Destruktor dipanggil ketika memori dibebaskan, hal ini terjadi setelah array objek digunakan dan memori yang dialokasikan untuk array tersebut dibebaskan dengan delete[], pesan "Destructor called!" muncul untuk setiap objek dalam array. Sehingga menunjukan bahwa destruktor kelas Box dipanggil dengan benar saat memori objek dilepaskan.

- Program berjalan dengan baik. Dengan kode keluar 0, menandakan bahwa tidak ada kesalahan yang fatal terjadi selama eksekusi progam

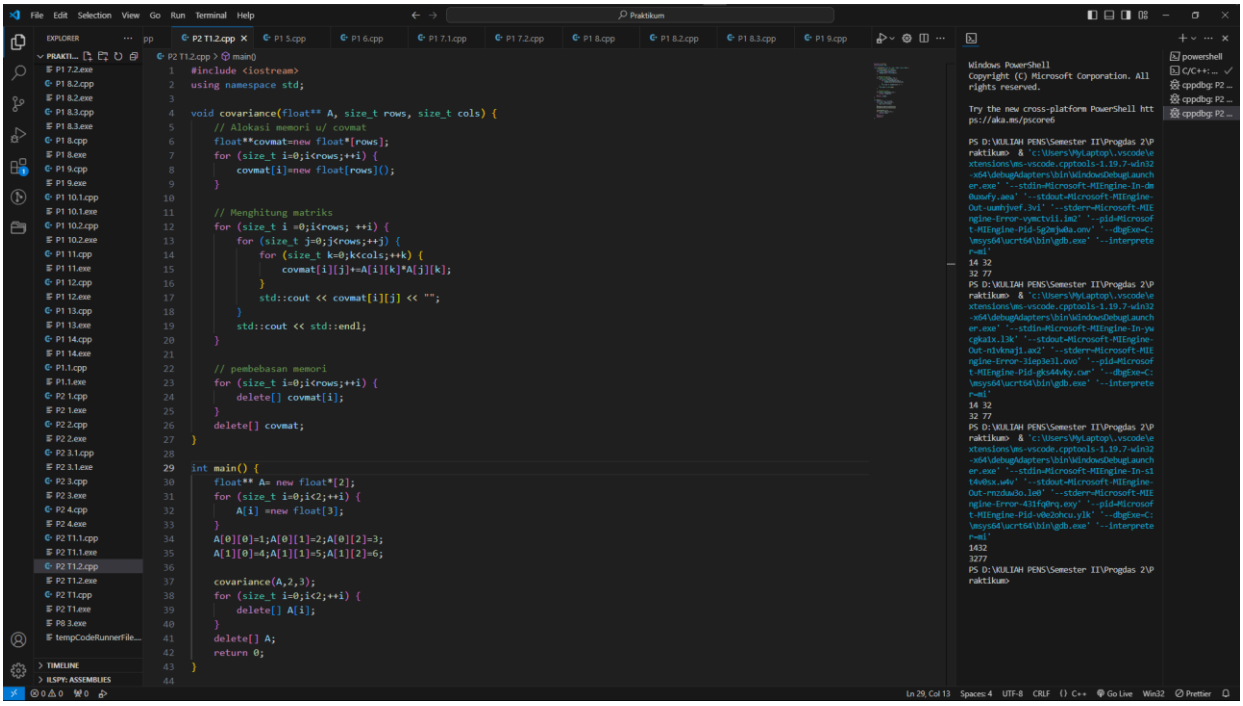
Tugas

- Buatlah sebuah program untuk menghitung matrikskovarian dari sebuah matriks berukuran 2 x 3.
- Program harus menggunakan alokasi memoridinamis dengan tipe data float.
- Nilai-nilai dalam matriks adalah sbb:A = [1 2 3; 4 5 6]
- Covariance matriks: $A.A^T$ (dot produk)
- Jika A berukuran 2 × 3, maka A^Tberukuran 3 × 2
- Hasil akhir dari perkalian matriks berukuran 2 × 3dan 3 × 2 adalah 2 × 2

Kode pemograman:

```
#include <iostream>
using namespace std;

void covariance(float** A, size_t rows, size_t cols) {
    // Alokasi memori u/ covmat
    float**covmat=new float*[rows];
    for (size_t i=0;i<rows;++i) {
        covmat[i]=new float[rows]();
    }

    // Menghitung matriks
    for (size_t i =0;i<rows; ++i) {
        for (size_t j=0;j<rows;++j) {
            for (size_t k=0;k<cols;++k) {
                covmat[i][j]+=A[i][k]*A[j][k];
            }
            std::cout << covmat[i][j] << "";
        }
        std::cout << std::endl;
    }

    // pembebasan memori
    for (size_t i=0;i<rows;++i) {
        delete[] covmat[i];
    }
    delete[] covmat;
}

int main() {
    float** A= new float*[2];
    for (size_t i=0;i<2;++i) {
        A[i] =new float[3];
    }
```

```
    A[0][0]=1;A[0][1]=2;A[0][2]=3;
    A[1][0]=4;A[1][1]=5;A[1][2]=6;

    covariance(A,2,3);
    for (size_t i=0;i<2;++i) {
        delete[] A[i];
    }
    delete[] A;
    return 0;
}
```



Outputnya ialah matriks 2x2
14 32
32 77